

Design Patterns in Python

Observer Pattern

Definition:

The Observer pattern defines a one-to-many relationship between objects.

When the Subject changes state, all registered Observers are notified.

How it works:

- Subject holds state and manages a list of observers.
- Observers implement an update method to react.
- When Subject changes, it notifies all observers.

```
class Subject:
    def __init__(self):
        self._observers = []
        self._state = None
    def attach(self, obs):
        self._observers.append(obs)
    def detach(self, obs):
        if obs in self._observers:
            self._observers.remove(obs)
    def notify(self):
        for obs in self._observers:
            obs.update(self)
    def set_state(self, new_state):
        self._state = new_state
        self.notify()
    def get_state(self):
        return self._state

class Observer:
    def update(self, subject):
        print(f"Notification: {subject.get_state()}")
```

```
# Usage
celebrity = Subject()
fan1 = Observer(); fan2 = Observer()
celebrity.attach(fan1); celebrity.attach(fan2)
celebrity.set_state("Dropped a new song!")
```

Scenario-based Q&A:

Q: Weather station updates displays?

A: Subject = WeatherStation, Observers = Displays.

Q: Stock tracker notifies investors?

A: Subject = Stock, Observers = Investors.

Singleton Pattern

Code

Definition:

The Singleton pattern ensures a class has only one instance across the program.

How it works:

- First creation stores the instance.
- Future calls reuse it.

Uses: Database connections, Loggers, Config managers.

```
class Logger:
    __instance = None
    def __new__(cls):
        if cls.__instance is None:
            print("Creating Logger instance...")
            cls.__instance = super().__new__(cls)
            cls.__instance.log_messages = []
        return cls.__instance
    def log(self, message):
        self.log_messages.append(message)
        print(f"[LOG]: {message}")
    def show_logs(self):
        for msg in self.log_messages:
            print("-", msg)

# Usage
log1 = Logger(); log2 = Logger()
log1.log("App started"); log2.log("User logged in")
print(log1 is log2)
```

Scenario-based Q&A:

Q: Why Singleton for DB connections?

A: To share one connection.

Q: Why Singleton for logging?

A: To keep logs consistent in one file.

Adapter Pattern

Definition:

The Adapter pattern allows incompatible interfaces to work together.

How it works:

- Target: what client expects.
- Adaptee: has a different interface.
- Adapter: converts calls.

Code

- Client: uses Target unaware of Adaptee.

Analogy: Plug adapter makes US plug fit EU socket.

```
class MP3Player:
    def play_mp3(self, filename): pass

class MP4Player:
    def play_mp4(self, filename):
        print(f"Playing MP4: {filename}")

class MP4Adapter(MP3Player):
    def __init__(self, mp4):
        self.mp4 = mp4
    def play_mp3(self, filename):
        self.mp4.play_mp4(filename)

class MediaPlayer:
    def __init__(self, player: MP3Player):
        self.player = player
    def play(self, filename):
        self.player.play_mp3(filename)

# Usage
mp4 = MP4Player()
adapter = MP4Adapter(mp4)
media = MediaPlayer(adapter)
media.play("video.mp4")
```

Scenario-based Q&A:

Q: Old system uses OldPrinter.print_document(), new printer has print_file(). Solution?

A: Use PrinterAdapter.

Q: Game expects Keyboard.press_key(), but controller has press_button().

A: Use GamepadAdapter.

Code